UVC 是属于 USB 行业规范中的 USB 设备类规范，用于 USB 接口的视频设备的一个统一的数据交换规范。其中 UVC 官方协议文档 UVC 1.5 Class specification 是对整个 UVC 协议的框架做了规范，USB_Video_Example 则规定了描述符等的数据结构和相关的一些其他相关规定。

1、UVC 概述

在 UVC 规范中明确要求，具有实际 UVC 功能的视频设备只要需要两种接口：一种为 VC Interface(视频控制接口)，VS Interface(视频流接口)。其中 VC 接口用于对 UVC 设备进行配置操控，而 VS 接口则用于负责传输视频数据流，两者相互配合完成 UVC 设备功能。

2、UVC 拓扑结构

UVC 的将设备在拓扑结构上抽象成为了几个主要的硬件功能模块，有以下几种：

输入端点(Input Terminal)

输出端点(Output Terminal)

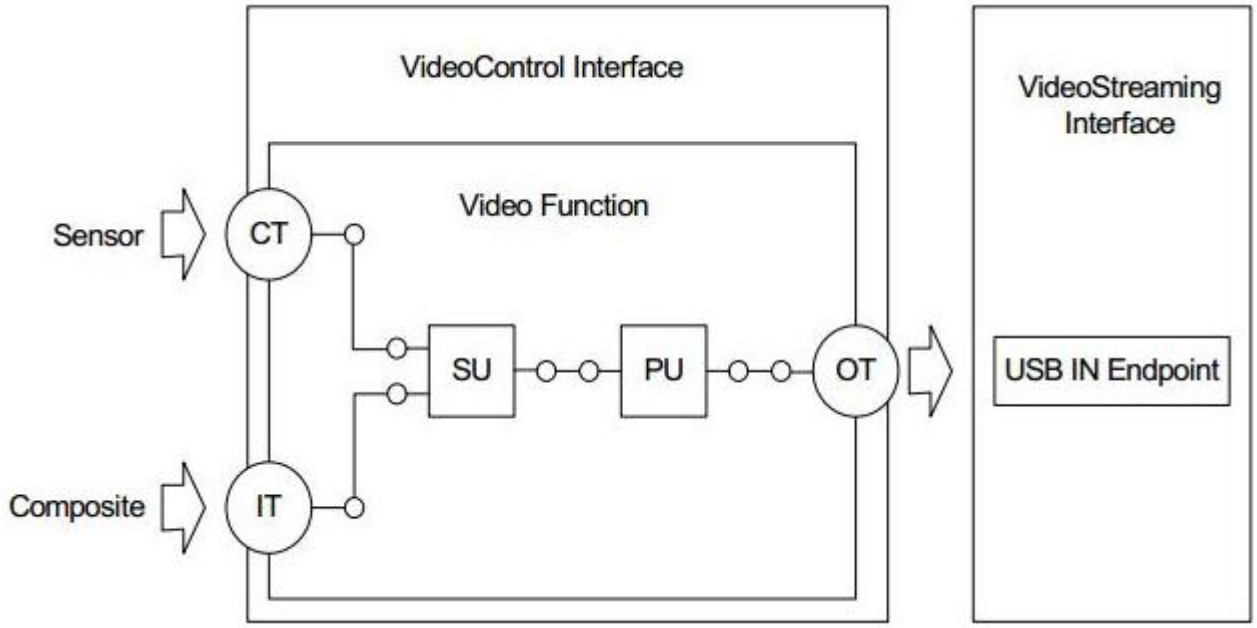
camera 端点(Camera Terminal)

选择单元(Selector Unit)

处理单元(Processing Unit)

拓展单元(Extension Unit)

在 USB_Video_Example 1.5 中以一个网络摄像头设备为例展现其拓扑结构的示例图如下：



从 Sensor 和另一个复合视频设备得到的数据流由 CT 和 IT 输入，经过 SU 选择送到 PU 处理，再由 OT 绑定到指定的 USB 端点。最后由 USB 端点与主机交互数据发送到 host。

在实际的 UVC 设备设备中，可能没有上图中的一些功能某块，可能只有其中一些模块来完成的。

同时从上面的拓扑结构图可以看出，左半部分框架组成了 UVC 中的 VC 接口部分，右半部分框架组成了 VS 接口部分。

3、概念补充

3.1 Terminal（端点）

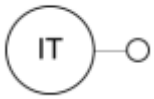
Terminal 是指整个 UVC 功能的入口和出口，逻辑意义上讲 Terminal 只有一个出口或者入口，它要么代表整个 UVC 功

能的数据输入点，要么代表数据在流经 UVC 内部的各 Unit 功能单元后的最终流出点。

1）IT(输入端点)

An Input Terminal (IT) is an entity that represents a starting point for data streams inside the video function.
一个输入 Terminal (IT)终端是一个实体，代表数据流的开始端点

输入端点一般在拓扑图中用下图表示：



2）OT(输出端点)

An Output Terminal (OT) represents an ending point for data streams.

一个输出 Terminal (OT)终端是一个实体，代表数据流的结束端点。

输出端点一般在拓扑图中用下图表示：



3）CT(Camera 端点)

CT 代表硬件上的数据采集源(sensor),用于控制视频流传输中的设备特性的。它规定支持以下特性控制：

Scanning Mode (Progressive or Interlaced)：扫描模式(逐行扫描或隔行扫描)

Auto-Exposure Mode：自动曝光模式

Auto-Exposure Priority：自动曝光优先级

Exposure Time：曝光时间

Focus：聚焦

Auto-Focus：自动聚焦

Simple Focus：简单聚焦

Iris：可变光圈

Zoom：缩放/变焦

Pan：移动镜头

Roll：滚动

Tilt：倾斜

Digital Windowing：数字窗口

Region of Interest：感兴趣区域

3.2 Unit(单元)

Unit 可以理解为构建出 UVC 设备功能的各功能单元，多个 Unit 按照一定的规则连接后就是一个完整的 UVC 功能设备。Spec 规定 Unit 有一个或多个入口，一个出口;也就是说可以有多个 Unit 或 Terminal 作为一个 Unit 的输入源头，而数据

在流经 Unit 后仅能作为其他 Unit 或 Terminal 的一个输入源头。

规范 1.5 中定义了四种 Unit：

Selector Unit (SU)，Processing Unit (PU)，Extension Unit (EU)，Extension （XU）

1)SU(选择单元)

SU 单元用来控制图像数据采集源的切换和选择。例如上面相机拓扑图有两个数据输入源，这时需要 SU 来进行切换和选择。选择单元一般在拓扑图中用下图表示：



2)PU(处理单元)

PU 代表对采集所得图像数据进行加工的单元，负责对采集到的图像特性进行调整设置，主要有以下一些特性：

User Controls

Brightness（背光）

Contrast（对比度）

Hue（色度）

Saturatio（饱和度）

Sharpness（锐度）

Gamma（伽马）

Digital Multiplier (Zoom)（数字放大）

Auto Controls

White Balance Temperature（白平衡色温）

White Balance Component（白平衡组件）

Blcklight Compensation（背光补偿）

Contrast（对比度）

Other

Gain（增益）

Power Line Frequency（功率线平率）

Analog Video Standard（模拟视频标准）

Analog Video Lock Status（模拟视频死锁状态）

处理单元一般在拓扑图中用下图表示：



3）EU(编码单元)

EU 用于对采集所得的数据进行个性化处理的功能。

编码单元一般在拓扑图中用下图表示：



4）XU(扩展单元)

XU 是为了特定供应商添加规范而提供的单元。

扩展单元一般在拓扑图中用下图表示：



3.3、Video Control Interface（视频控制接口）

VC 主要用于控制 UVC 功能中的 Units 和 Terminals，通过对不同的 Units 和 Terminals 下发不同的指令，从而影响 UVC 设备的当前工作状态;Host 端下发的所有对 UVC 设备的操纵控制命令都是经由此 Interface 完成的。以上面拓扑图所表示的设备为例：

该设备包含了一个 Processing Unit、一个 Input Terminal、Output Terminal 和 Selector Unit，VC Interface 需要对它们下发不同的命令，为此 VC Interface 需要包含一个控制端点(强制性要求)，它使用的就是每个 USB 设备中默认的端点 0。另外一个 Interrupt 端点则是可选的，用来返回或通知 Host 端当前的 UVC 设备内部状态有变化。大部分情况下，一个 UVC 设备的 VC Interface 不需一定要实现此端点，但一旦 UVC 设备需要实现某些特定 feature 时，Spec 会强制性要求实现该 interrupt 端点。

3.4、Video Streaming Interface（视频流接口）

VS Interface 则专注与负责传输 UVC 设备的 Video 数据到 Host 端。若一 UVC 设备支持 n 种格式的 Video 数据，则它需要实现 n 个 VS Interface，每个 Interface 对应一种专门的数据格式;

3.5、Video Interface Collection(视频接口集合)&&Interface Association Description(视频关联描述符)

View Code

视频接口集合

视频接口集合是用于管理同一功能中 VC 和 VS 接口的。如果设备包含多个独立的视频功能，那么就可能存在多个视频接口集合。

视频关联描述符

一个设备必须使用一个 IAD(接口关联描述符)去描述一个 Video Interface Collection(视频接口集合)，USB 协议中并没有专门朝设备索取 IAD 的命令，AD 需作为 Configuration 描述符的一部分发送给 Host 端(返回 GetDescriptor(Configuration)请求的回应)。在该 Configuration 描述符的排列顺序中，IAD 要放在它所包含的 VC,VS 描述符之前，且同一个 IAD 中所有的 VC,VS Interface 的 Interface number 必须连贯。

4、Descriptor（描述符）

描述符用来描述 USB 设备性能或特向的数据结构，与设备类相关的信息都是主机向设备获取描述符来得到的。

在 UVC 所规定的描述符中除了常用的标准描述符，另外还定义了视频设备的特殊类描述符，主要如下：

标准描述符

- 设备描述符（Device Descriptor）
- 设备限定描述符（Device QualifierDescriptor）
- 设备配置描述符（Configure Descriptor）
- 其他速度描述符（Other Speed Descriptor）
- 字符描述符（String Descriptor）

特殊类描述符

- 接口联合描述符（Interface Association Descriptor）
- 视频控制接口描述符（VideoControl Interface Descriptor）
- 视频控制端点描述符（VideoControl Endpoint Descriptor）
- 视频流接口描述符（VideoStreaming Interface Descriptor）
- 视频流端点描述符（VideoStreaming Endpoint Descriptor）

5、Request（请求）

请求是主机软件与实体中控制器交流的一种机制。请求都是由主机向设备端发起的功能请求，所有的 USB 设备都需要支持标准的设备请求和与 Class 相关的特殊类请求。

标准请求

在标准协议中有相关描述，不做详细展开

特殊类请求

事实上特殊类请求也可以根据接口来分成两大类：1）、视频控制请求，2）视频流请求。

1）视频控制请求有：

- 接口控制请求（Interface Control Request）
- Camera 终端控制请求（Camera Terminal Control Request）
- 选择单元控制请求（Selector Unit Control Request）
- 处理单元控制请求（Processing Unit Control Request）
- 扩展单元控制请求（Extension Unit Control Request）

2）视频流请求仅有接口控制请求(Interface Control Request)一种：

Interface Control Requests 因为是用来在主机和设备之间协商数据交互格式和分辨率，流量等信息的，所以一般来说是必须实现的。

Camera Terminal Control Requests 包含了对曝光时间，曝光模式，对焦，变焦，平移等 sensor 获取数据阶段时的参数控制。

Processing Unit Control Requests 中则包含了亮度，增益，色调，对比度，白平衡等等 sensor 在获取到图像数据后，图像处理阶段的相关参数。

不过实际上，以上两者的划分在硬件层次并不是绝对的。很多参数的控制在 sensor 硬件级别上是同一层次的。不过，将这些功能抽象成这两类，正如在硬件的拓扑结构上将功能模块抽象出来一样，有利于通用化的程序设计。

AndroidUVCCamera

最近在做一个外接 USB 相机的项目，github 上搜了下，有很多开源的库，这些库底层基本用的都是同一套东西，但上层业务比较繁琐，使用起来很不方便，并且也不太符合项目的业务需求，所以重新封装了一下，本库只需几个简单的 API 即可完成预览、拍照功能，无需关注复杂的 USB 插拔处理逻辑。

1.添加依赖

Step 1. Add the JitPack repository to your build file.Add it in your root build.gradle at the end of repositories:

```
allprojects {
  repositories {
    ...
    maven { url 'https://jitpack.io' }
  }
}
```

Step 2. Add the dependency

```
dependencies {
        implementation 'com.github.Liuguihong:AndroidUVCCamera:1.0.0'
}
```

2.创建 UVCCameraProxy 对象

```
UVCCameraProxy mUVCCamera = new UVCCameraProxy(this);
```

3.添加配置（可选）

```
mUVCCamera.getConfig()
         .isDebug(true) // 是否调试
         .setPicturePath(PicturePath.APPCACHE) // 图片保存路径，保存在 app 缓存还是 sd 卡
         .setDirName("uvccamera") // 图片保存目录名称
         .setProductId(0) // 产品 id，用于过滤设备，不需要可不设置
         .setVendorId(0); // 供应商 id，用于过滤设备，不需要可不设置
```

4.设置预览 View

支持 TextureView 和 SurfaceView，在其生命周期里封装了注册/注销 USB 插拔广播、释放相机资源等逻辑，并且自动过滤 USB 设备，只响应 USB 相机插拔。

```
mUVCCamera.setPreviewTexture(mTextureView); // TextureView
// mUVCCamera.setPreviewSurface(mSurfaceView); // SurfaceView
```

5.设置 USB 监听回调，并在回调里处理相关逻辑

```
mUVCCamera.setConnectCallback(new ConnectCallback() {
    @Override
    public void onAttached(UsbDevice usbDevice) {
        mUVCCamera.requestPermission(usbDevice); // USB 设备授权
    }

    @Override
    public void onGranted(UsbDevice usbDevice, boolean granted) {
        if (granted) {
            mUVCCamera.connectDevice(usbDevice); // 连接 USB 设备
        }
    }

    @Override
    public void onConnected(UsbDevice usbDevice) {
        mUVCCamera.openCamera(); // 打开相机
    }

    @Override
    public void onCameraOpened() {
        mUVCCamera.setPreviewSize(640, 480); // 设置预览尺寸
```

```
        mUVCCamera.startPreview(); // 开始预览
    }

    @Override
    public void onDetached(UsbDevice usbDevice) {
        mUVCCamera.closeCamera(); // 关闭相机
    }
});
```

6.设置拍照按钮点击回调（可选）

```
mUVCCamera.setPhotographCallback(new PhotographCallback() {
    @Override
    public void onPhotographClick() {
        mUVCCamera.takePicture();
    }
});
```

7.设置预览回调（可选）

```
mUVCCamera.setPreviewCallback(new PreviewCallback() {
    @Override
    public void onPreviewFrame(byte[] yuv) {

    }
});
```

8.设置拍照成功图片回调（可选）

```
mUVCCamera.setPictureTakenCallback(new PictureCallback() {
    @Override
    public void onPictureTaken(String path) {

    }
});
```

9.拍照

```
mUVCCamera.takePicture();
// mUVCCamera.takePicture("test.jpg"); // 自定义图片名称，不设置则根据 UUID 自动保存
```

10.其他 API

方法	说明
registerReceiver()	注册 USB 插拔监听广播
unregisterReceiver()	注销 USB 插拔监听广播
checkDevice()	查找 USB 相机设备，会在 onAttached 里回调
requestPermission(UsbDevice usbDevice)	USB 设备授权，要连接 USB 设备必须先授权
connectDevice(UsbDevice usbDevice)	连接 USB 设备
closeDevice()	关闭 USB 设备
openCamera()	打开相机
closeCamera()	关闭相机

方法	说明
setPreviewSurface(SurfaceView surfaceView)	设置预览 View 为 SurfaceView
setPreviewTexture(TextureView textureView)	设置预览 View 为 TextureView
setPreviewDisplay(Surface surface)	设置预览 View，自定义
setPreviewRotation(float rotation)	设置相机预览旋转角度，暂时只支持 TextureView
setPreviewSize(int width, int height)	设置预览尺寸
getPreviewSize()	获取相机预览尺寸
getSupportedPreviewSizes()	获取相机支持的预览尺寸
startPreview()	开始预览
stopPreview()	停止预览
takePicture()	拍照
takePicture(String pictureName)	拍照
isCameraOpen()	是否已经打开相机
getConfig()	获取配置信息
clearCache()	删除图片缓存目录

参考

https://github.com/saki4510t/UVCCamera